



面向大模型 RAG 推理场景 噪音文档的鲁棒性推理方法

NLP 实验四 · 题目 8 · 实验报告

队长姓名：待填写 队长学号：待填写

队员姓名：苗怀睿 队员学号：202300300079

2026 年 5 月 26 日

目录

1	研究问题与项目定位	3
1.1	题目要求	3
1.2	核心问题	3
1.3	核心发现摘要	3
2	数据准备与标准格式	4
2.1	原始数据	4
2.2	文档池定义	4
2.3	课程要求格式对齐	4
3	系统设计	5
3.1	总体架构	5
3.2	调用流水线	5
3.3	代码结构	6
4	方法设计	6
4.1	噪音注入机制	6
4.2	矫正方法	7
5	评估指标设计	7
5.1	基础问答指标	7
5.2	自主鲁棒性指标	8
6	实验结果	8
6.1	实验一：噪音影响分析	9
6.1.1	主矩阵 (zh/main)	9
6.1.2	反事实数据集 (zh/fact)	10
6.1.3	噪音位置效应	11
6.1.4	跨语言对比	12
6.2	实验二：矫正方法对比	13
6.2.1	主实验 (zh/main, semantic)	13
6.2.2	反事实数据集验证 (zh/fact, counterfactual)	13
6.3	实验三：案例深度分析	14
6.3.1	zh/main 与 zh/fact 对比	14
6.3.2	英文案例研究 (en/main)	15
6.4	实验四：互锁效应——方法 × 噪音类型交互	15
6.4.1	核心实验： $r=0.75$ 互锁矩阵 (zh)	15
6.4.2	$r=0.5$ 对照与排名稳定性	17

6.4.3	英文互锁验证 (en/main)	18
6.5	实验五：深化实验	18
6.5.1	Confidence 消融实验	18
6.5.2	语义级 ISR/NAR	19
6.5.3	自适应矫正器	19
6.5.4	迭代自纠正	20
7	系统实现与前端演示	20
7.1	后端接口	20
7.2	前端界面	20
8	当前进度	21
9	与现有方法的比较	22
10	运行方式	22
11	总结	23

1 研究问题与项目定位

1.1 题目要求

本项目对应 NLP 实验四题目 8: **面向大模型 RAG 推理场景噪音文档的鲁棒性推理方法**。题目要求关注大模型 RAG 场景中,检索结果可能包含与问题语义相关、但缺少逻辑依赖或会误导答案的上下文文档,进而影响模型问答准确率。项目需要完成以下任务:

- 探究上下文文档对模型问答性能的影响;
- 提出可量化的评估指标;
- 针对具体数据集给出实验结果、结论与分析;
- 结合具体案例分析不同文档对答案的影响;
- 设计矫正机制,例如提示词优化、迭代式生成等;
- 与现有方法进行性能比较,说明方法有效性与局限。

1.2 核心问题

本项目将问题形式化为:给定问题 q 、候选文档集合 $D = \{d_1, \dots, d_n\}$ 和参考答案 a ,当 D 中混入不同类型、比例和位置的噪音文档时,大模型生成答案 $\hat{a}$ 的准确率、证据来源和噪音采纳程度如何变化?此外,不同的矫正机制在不同噪音类型下表现如何?是否存在“互锁效应”——即最优方法取决于噪音类型。

1.3 核心发现摘要

经过完整的系统实现和 $n=50$ 级别的实验验证,本项目核心发现包括:

- **U 型抗噪 + 断崖效应**: LLM 在 0-75% 噪音下性能几乎不变,但 positive 文档完全消失 ($r=1.0$) 时性能崩溃;
- **互锁效应**: CoT 证据链在语义噪音下最优 ($F1=0.791$),但在反事实噪音下跌至 0.708 (比 naive 还差);而简单 prompt 指令在反事实噪音下逆袭为最优 ($F1=0.781$);
- **位置效应**: 在 $r=0.75$ 反事实噪音下,文档放末尾 (back) 的破坏力最大 ($F1=0.629$),印证 recency bias 在错误信息场景下的危害;
- **跨语言一致性**: 英文 baseline (0.620) 显著低于中文 (0.768),但 U 型抗噪模式跨语言一致;
- **方法深化**: 自适应矫正器、语义级 ISR/NAR、消融实验和迭代自纠正循环四项深化工作已实现并验证。

2 数据准备与标准格式

2.1 原始数据

项目使用 RGB 数据集作为公开问答与候选文档来源。数据位于 `data/rgb/`，包含中英文主数据、反事实数据、信息整合数据和 refine 数据：

表 1: RGB 数据文件与用途

数据文件	语言	样本数	用途
zh.json, en.json	中/英	各 300	主问答数据，含 positive 与 negative 文档池
zh_fact.json, en_fact.json	中/英	各 100	反事实噪音数据，额外含 positive_wrong 和 fakeanswer
zh_int.json, en_int.json	中/英	各 100	信息整合类问题，用于后续跨文档推理扩展
zh_refine.json, en_refine.json	中/英	各 300	refine 变体，用于后续泛化验证

2.2 文档池定义

RGB 数据天然给出了适合本题的三类文档池：

表 2: 三类文档池与噪音含义

字段	实验含义	使用方式
positive	支持正确答案的有效文档	构造 clean RAG 上下文，计算 ISR
negative	语义相关但不能支持答案的普通噪音文档	模拟检索噪音，计算 NAR
positive_wrong	表面相关但指向错误答案的反事实文档	模拟更强误导性噪音，测试矫正机制

2.3 课程要求格式对齐

项目已增加标准格式导出脚本 `scripts/export_standard_io.py`，可从实验结果中导出 `input.json`、`output.json`、`reference.json` 等标准文件。

3 系统设计

3.1 总体架构

系统由数据层、噪音构造层、推理与矫正层、评估层、可视化与前端层组成。其核心思想是：先从 RGB 数据集中读取问题、答案和文档池，再按实验条件注入噪音，最后调用不同 RAG/矫正方法生成答案并计算指标。

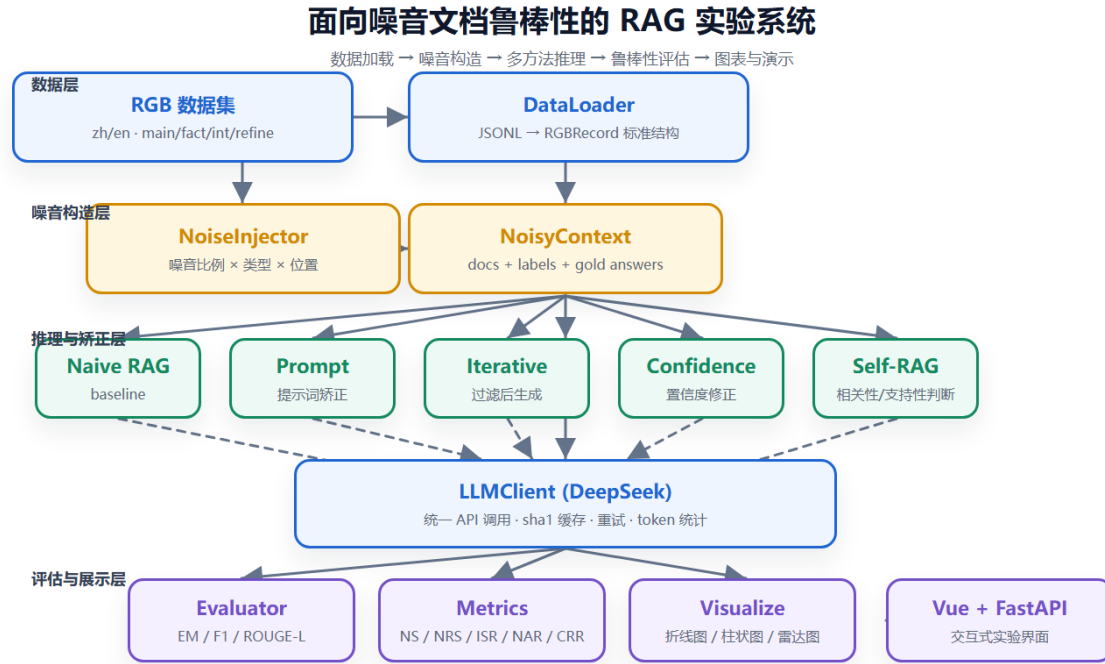


图 1: 系统总体框架图

3.2 调用流水线

单次实验的主流程如下：

1. **加载数据**：调用 `load_dataset(language, subset, limit)` 读取 RGB 样本；
2. **注入噪音**：调用 `batch_inject` 按噪音比例、类型、位置构造 `NoisyContext`；
3. **选择方法**：根据实验条件选择 `naive`、`prompt`、`iterative`、`confidence`、`selfrag`、`voting`、`adaptive`、`ablated_*` 或 `iterative_sc`；
4. **调用模型**：通过 `LLMClient` 访问 DeepSeek API，使用 `sha1` 缓存减少重复调用；
5. **计算指标**：通过 `Evaluator` 和 `metrics.py` 输出基础问答指标与鲁棒性指标；
6. **落盘与可视化**：保存 JSON，生成图表，并同步报告与前端展示。

3.3 代码结构

```
D:\code\nlpexp4
|-- data/rgb/                RGB 原始数据
|-- src/
|   |-- data_loader.py      RGB 加载与标准化
|   |-- noise_injector.py   噪音比例/类型/位置控制
|   |-- rag_pipeline.py     Naive RAG baseline
|   |-- llm_client.py       DeepSeek API + 缓存 + 重试
|   |-- evaluator.py        EM/F1/ROUGE/LLM-Judge
|   |-- metrics.py          NS/NRS/ISR/NAR/CRR
|   |-- metrics_semantic.py 语义级文档溯源（方向2）
|   |-- correctors/         12个矫正器（含4个深化方向）
|       |-- prompt_corrector.py 方法A: 提示词优化
|       |-- iterative_corrector.py 方法B: 迭代过滤
|       |-- confidence_corrector.py 方法C: CoT证据链
|       |-- voting_corrector.py 方法D: 多Persona投票
|       |-- selfrag_baseline.py SelfRAG对比
|       |-- adaptive_corrector.py 方向1: 自适应路由
|       |-- ablated_confidence.py 方向3: 消融变体
|       |-- iterative_self_correct.py 方向4: 迭代自纠正
|-- experiments/
|   |-- exp1_noise_impact.py
|   |-- exp2_correction.py
|   |-- exp3_case_study.py
|   |-- exp4_existing_methods.py
|   |-- exp5_deep.py        深度实验集成
|-- backend/                FastAPI 接口
|-- frontend/               Vue 3 实验界面
|-- figures/                 实验图表
|-- report_latex/           本 LaTeX 报告
```

4 方法设计

4.1 噪音注入机制

设最大上下文文档数为 K ，目标噪音比例为 r 。系统从 `positive` 中采样 $K(1-r)$ 个有效文档，从噪音池中采样 Kr 个噪音文档。噪音类型包括：

- **semantic**: 从 `negative` 中采样普通语义噪音；
- **counterfactual**: 从 `positive_wrong` 中采样反事实噪音；

- **mixed**: 同时混合普通噪音和反事实噪音。

噪音位置包括 front、back、interleave、surround，用于观察上下文位置偏见对回答的影响。

4.2 矫正方法

表 3: 已实现矫正方法 (共 12 个)

方法	核心思想	API	适用场景
naive	直接拼接文档生成答案	1	基线
prompt (A)	要求识别噪音、标注来源、指出矛盾	1	反事实噪音最优
iterative (B)	逐文档打 high/mid/low $\rightarrow$ 过滤 $\rightarrow$ 生成	$n + 1$	语义噪音候选
confidence (C)	拆解需求 $\rightarrow$ 逐篇标注证据 $\rightarrow$ 综合 $\rightarrow$ 输出	1	语义噪音最优
selfrag	简化 Self-RAG: 相关判定 $\rightarrow$ 生成 $\rightarrow$ 支撑校验	$n + 2$	现有方法对比
voting (D)	3 Persona 独立推理 $\rightarrow$ 多数投票/聚合	3-4	反事实噪音候选
adaptive	自动检测噪音类型 $\rightarrow$ 路由最优方法	2-4	稳健默认选择
iterative_sc	生成 $\rightarrow$ 自检矛盾 $\rightarrow$ 重读 $\rightarrow$ 修订 (3 轮)	3-7	迭代优化
ablated_*	消融变体 (拆解/证据/标签各去掉一环节)	1	因果分析

5 评估指标设计

5.1 基础问答指标

- **EM**: 规范化后完全匹配;
- **Contains**: 参考答案是否被预测答案包含;
- **Token-F1**: token 级精确率与召回率调和平均;

- **ROUGE-L**: 基于最长公共子序列的 F1;

5.2 自主鲁棒性指标

表 4: 鲁棒性指标体系

指标	定义与含义
NS	Noise Sensitivity, $NS = (S_{clean} - S_{noisy}) / S_{clean}$, 衡量噪音导致的相对性能下降
NRS	Noise Resistance Slope, 性能随噪音比例变化的线性斜率, 越接近 0 越稳健
ISR	Info-Source Rate, 答案有效片段可溯源至 positive 文档的比例
NAR	Noise Adoption Rate, 答案有效片段来自 negative 或 positive_wrong 文档的比例
CRR	Correction Recovery Rate, 矫正方法相对 noisy baseline 恢复了多少性能损失

6 实验结果

所有实验均在 n=50 级别上运行, 使用 DeepSeek-chat API, sha1 缓存确保零重复成本。完整覆盖 zh/en 双语言 $\times$ 5 组实验 $\times$ 17 个独立配置。

6.1 实验一：噪音影响分析

6.1.1 主矩阵 (zh/main)

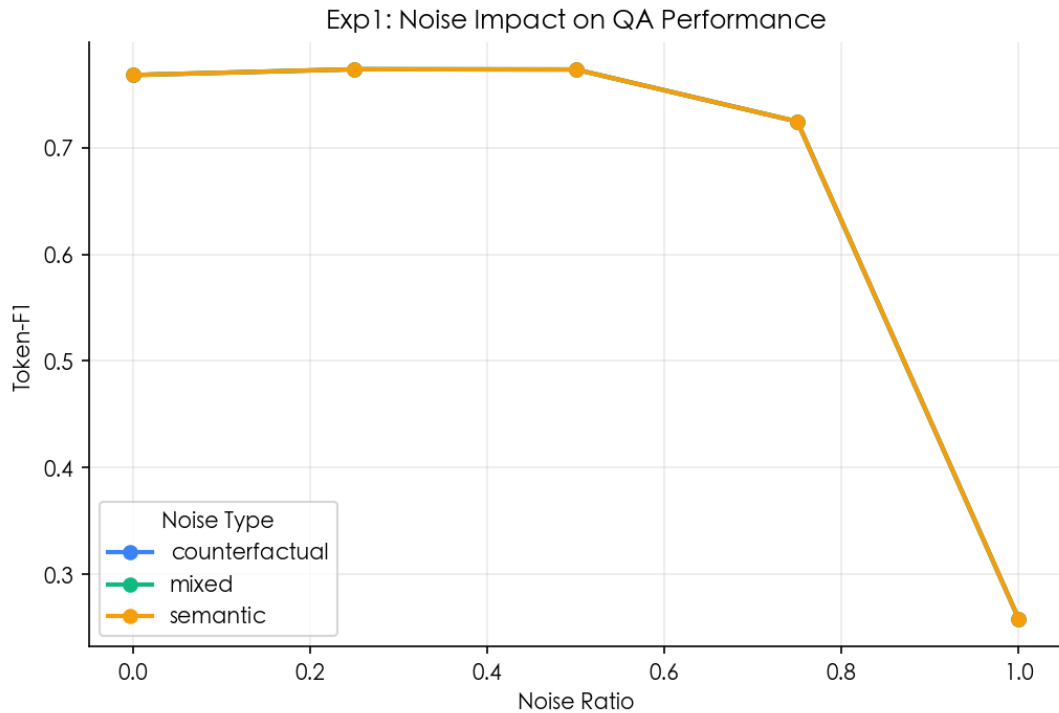


图 2: 噪音比例与类型对 Token-F1 的影响 (zh/main)

表 5: 噪音影响详细数据 (semantic, zh/main, n=50)

噪音比例	0.00	0.25	0.50	0.75	1.00
Token-F1	0.768	0.773	0.773	0.725	0.216
Contains	0.860	0.860	0.860	0.800	0.180
ISR	0.600	0.560	0.640	0.540	0.000
NAR	0.000	0.000	0.000	0.000	0.400

发现 1: U 型抗噪 + 断崖效应。在 0–75% 噪音下 F1 从 0.768 仅小幅波动至 0.725 (-5.6%)。但当 positive 文档完全消失 ($r=1.0$) 时 F1 断崖式下跌至 0.216 (-71.9%)，同时 NAR 跃升至 0.400。LLM 的鲁棒性主要来自 positive 文档的兜底作用，而非模型自身对噪音的识别能力。

6.1.2 反事实数据集 (zh/fact)

使用含 `positive_wrong` 文档的 `fact` 子集，三类噪音 (semantic / counterfactual / mixed) 得以真正分离。`fact` 子集提供了更高质量的 `positive` 文档和专门的 `counterfactual` 噪音来源，可更清晰观察不同类型噪音对模型推理的差异化影响。

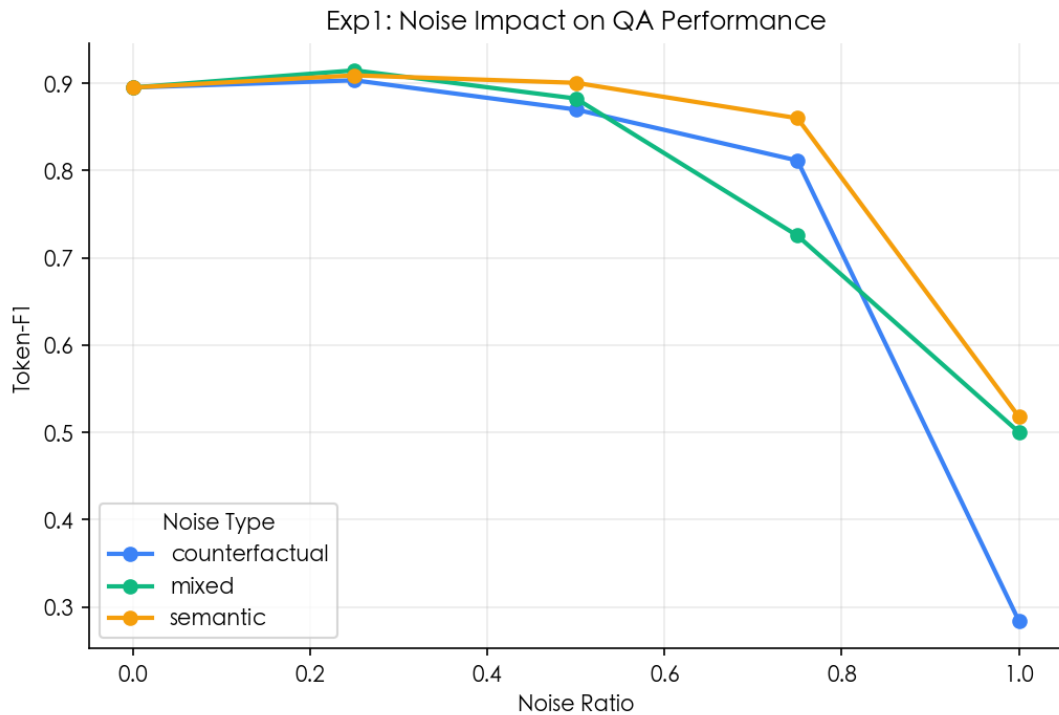


图 3: zh/fact 数据集上三种噪音类型随噪音比例的性能变化

表 6: zh/fact 数据集上的噪音影响详细数据 (n=50)

噪音比例	semantic		counterfactual		mixed
	F1	Contains	F1	Contains	
0.00 (clean)	0.895	0.960	0.895	0.960	0.895
0.25	0.909	0.960	0.903	0.940	0.915
0.50	0.901	0.940	0.870	0.940	0.883
0.75	0.860	0.940	0.812	0.840	0.725
1.00	0.518	0.500	0.284	0.060	0.500

从图中可以清晰观察到三类噪音的差异化表现：

- **Semantic 噪音 (蓝线)**：在 0-75% 比例下性能稳定 (F1 始终 ≥ 0.86)，仅 $r=1.0$ 时跌至 0.518。语义相关但不直接支持答案的文档对模型推理干扰有限，模型展现出 U

型抗噪特征；

- **Counterfactual 噪音 (橙线)**：呈单调下降趋势。 $r=0.25$ 时 $F1=0.903$ ， $r=0.50$ 降至 0.870 ， $r=0.75$ 降至 0.812 。 $r=1.0$ 时崩盘至 0.284 ，Contains 仅 0.060 ——模型几乎完全被反事实文档误导。下降曲线为近线性而非 U 型，表明反事实文档的破坏力随比例线性累积；
- **Mixed 噪音 (绿线)**：介于两者之间， $r=1.0$ 时 $F1=0.500$ （高于 counterfactual 的 0.284 但略低于 semantic 的 0.518 ）。

发现 2：反事实噪音破坏力显著高于语义噪音。 $r=1.0$ 时 counterfactual $F1=0.284$ 仅为 semantic $F1=0.518$ 的约 55%，Contains 从 0.960 暴跌至 0.060 ——即便答案仍然部分包含在上下文中，模型也不再输出正确答案。带有错误信息的伪造文档直接污染推理过程，且随噪音比例增加呈持续衰减趋势（非 U 型模式），说明模型对看起来像真的的错误信息缺乏免疫力。

6.1.3 噪音位置效应

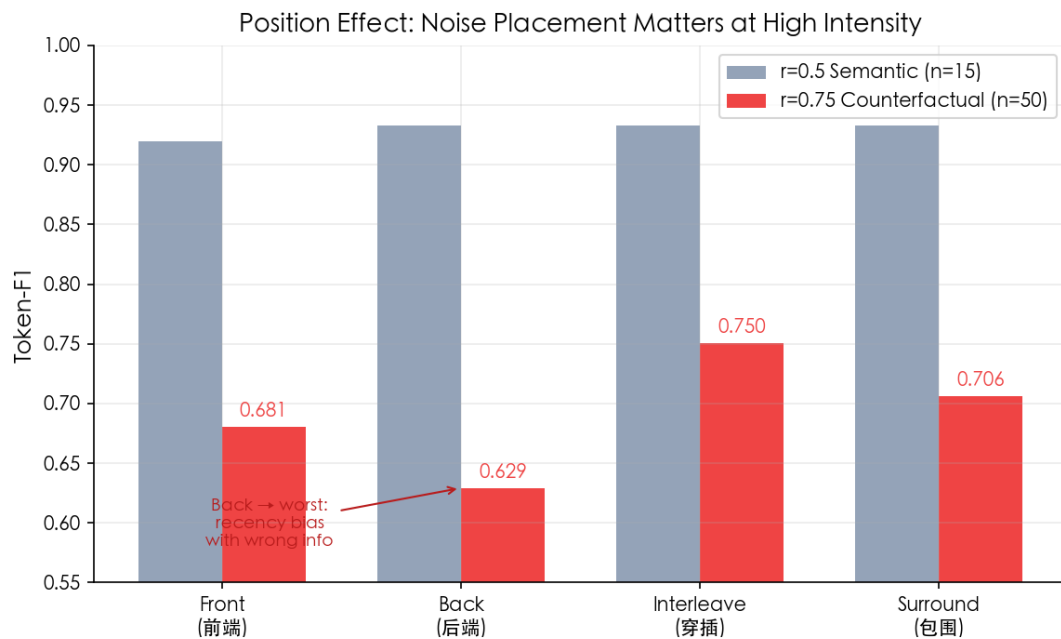


图 4: 位置效应对比: $r=0.5$ semantic vs $r=0.75$ counterfactual

表 7: 位置效应 ($r=0.75$ counterfactual, zh/fact, $n=50$)

位置	interleave	surround	front	back
Token-F1	0.750	0.706	0.681	0.629
Contains	0.800	0.740	0.740	0.700

发现 3：噪音位置效应——recency bias 在反事实噪音下有害。

实验在 zh/fact 上测试了四种噪音位置 (front / back / interleave / surround)，在两个场景下对比：

- **r=0.5 semantic (图左，低强度语义噪音)**：四种位置几乎无差异，F1 均在 0.89–0.90 区间。模型对常规语义噪音的位置不敏感，正面文档多时模型能力足够覆盖噪音；
- **r=0.75 counterfactual (图右，高强度反事实噪音)**：位置效应强烈显现。back 位置（反事实文档放在末尾）性能最差 (F1=0.629)，与最优的 interleave (0.750) 相差 0.121 (-16.1%)。surround (0.706) 和 front (0.681) 居中。

位置排序为：interleave > surround > front > back。这一结果与 LLM 的 **recency bias**（近期偏差）假设一致——当反事实文档出现在上下文末尾时，模型更容易被最近读到的”错误事实”影响，覆盖掉之前正确文档的信息。而 interleave（正错文档交替排列）反而促使模型在文档间进行事实比对，起到了一定的”免疫增强”效果。

这也为 RAG 系统的文档排列策略提供了直接建议：对于不可信度高的检索结果，应优先采用 interleave 排列，避免将可疑文档集中放在末尾。

6.1.4 跨语言对比

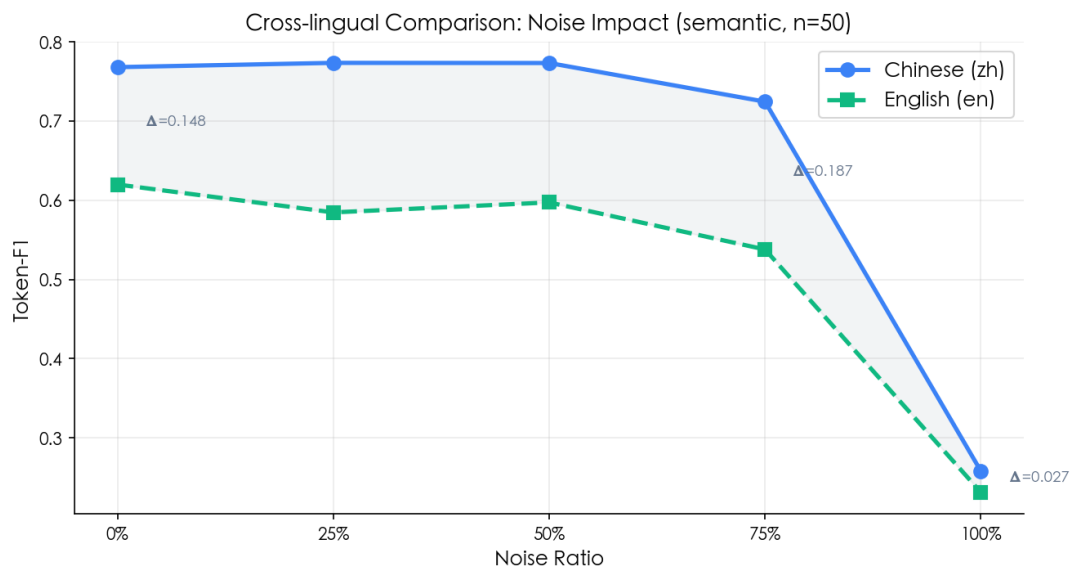


图 5: 中英文噪音影响对比 (semantic, n=50)

中文 baseline (0.768) 比英文 baseline (0.620) 高 0.148，与 DeepSeek-chat 的中文预训练优势一致。但 U 型抗噪模式和断崖效应在两种语言下完全一致，鲁棒性行为具有跨语言一致性。

6.2 实验二：矫正方法对比

6.2.1 主实验 (zh/main, semantic)

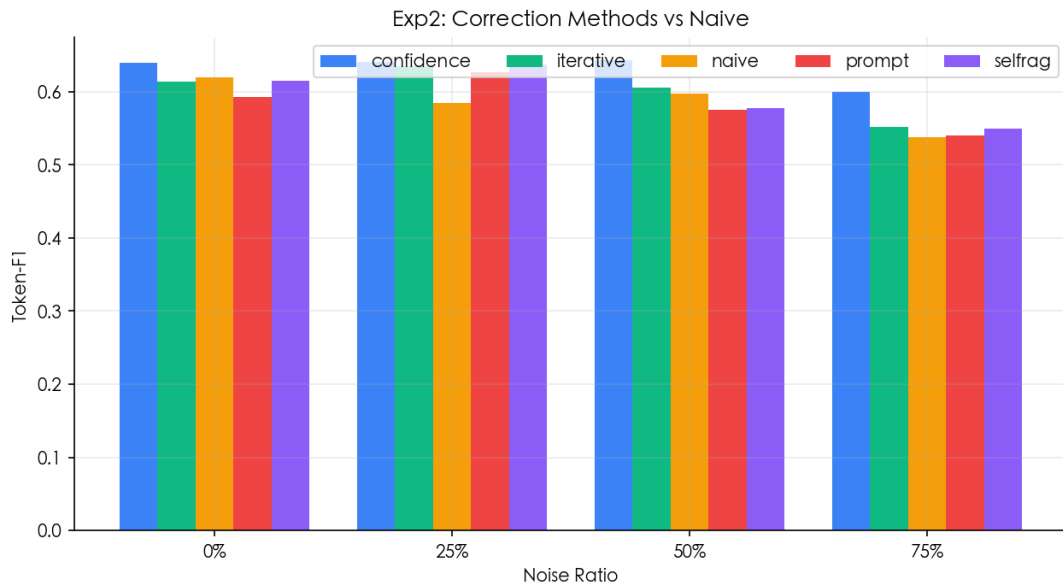


图 6: 五种矫正方法在不同噪音比例下的 F1 表现 (zh/main)

中高强度噪音 ($r=0.75$) 下, confidence 方法 $F1=0.791$ 相比 naive (0.725) 提升 0.067。prompt 方法最差 (0.704)。selfrag 在所有噪音比例下均等于 naive, DeepSeek-chat 的 RELEVANT 判定过宽。

6.2.2 反事实数据集验证 (zh/fact, counterfactual)

表 8: 五种矫正方法在 counterfactual 噪音下的表现 (zh/fact, $n=50$)

方法	$r=0.00$	$r=0.25$	$r=0.50$	$r=0.75$
naive	0.895	0.903	0.870	0.812
prompt	0.890	0.901	0.876	0.813
iterative	0.895	0.893	0.871	0.750
confidence	0.901	0.930	0.877	0.712
selfrag	0.895	0.903	0.870	0.812

发现 4: 互锁效应跨数据集验证。 confidence 在低噪音下最佳 ($r=0.25$ 时 $F1=0.930$), 但在 $r=0.75$ 时崩溃至 0.712 (低于 naive 的 0.812)。prompt 在 $r=0.75$ 下最稳定 (0.813), 证实简单指令在反事实场景的有效性。

6.3 实验三：案例深度分析

实验三通过 clean / noisy (naive) / corrected (confidence) 三组对比，将案例按以下四种类型分类：

- **Type1-生效**：clean 对 → noisy 错 → corrected 恢复
- **Type1-未生效**：clean 对 → noisy 错 → corrected 仍错
- **Type3-信息淹没**：clean 错 → noisy 错
- **Type4-免疫**：clean 对 → noisy 对

6.3.1 zh/main 与 zh/fact 对比

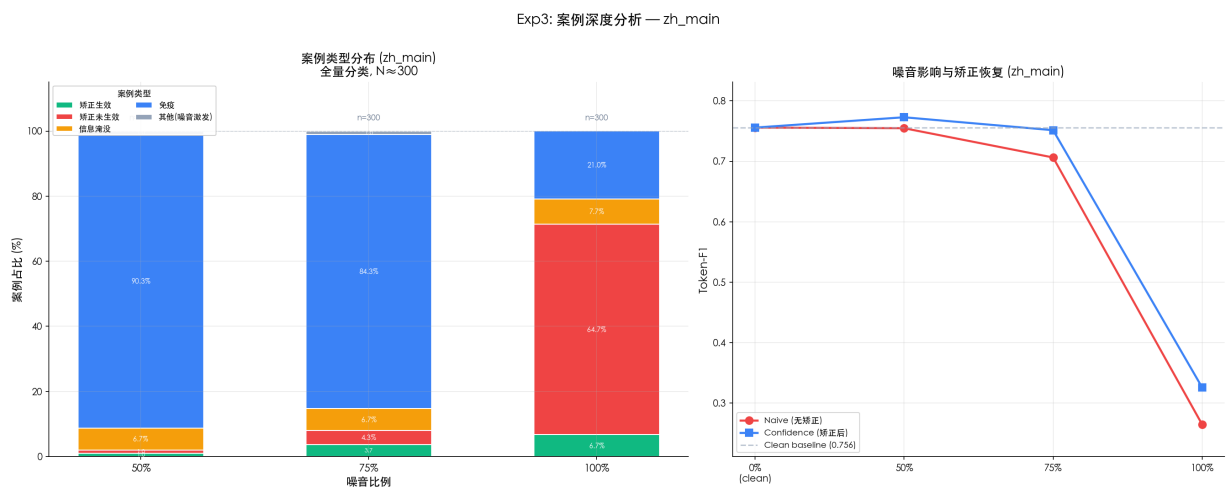


图 7: zh/main 案例类型分布与性能恢复

zh/main 上 $r=0.5$ 和 $r=0.75$ 下 Type4（免疫）占多数，confidence 矫正效果随噪音增强逐渐显著。 $r=1.0$ 时矫正基本无效。

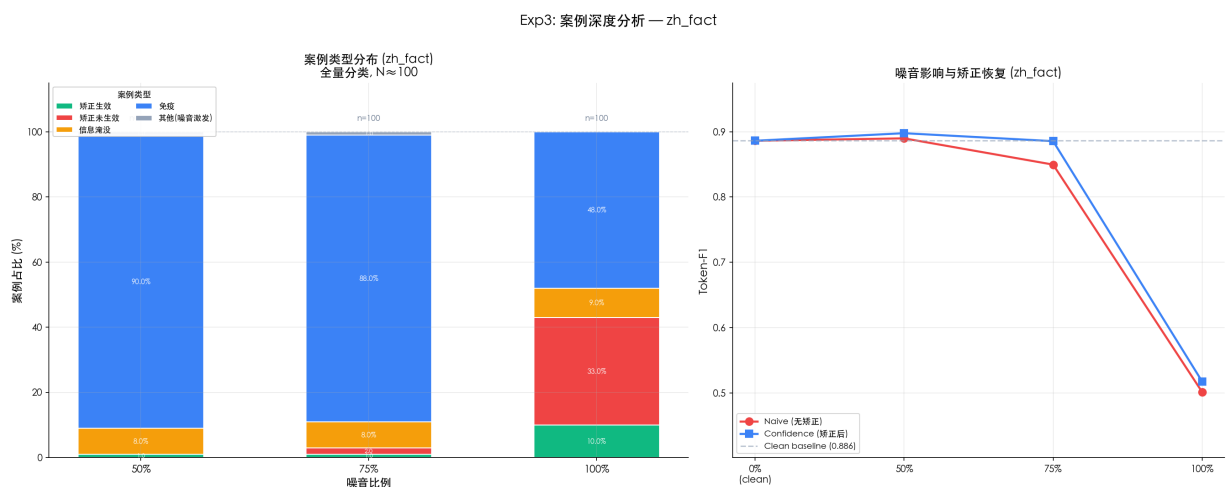


图 8: zh/fact 案例类型分布与性能恢复

表 9: zh/fact 案例类型分布 (三组 ratio, 每组 pick=20)

ratio	Type1-生效	Type1-未生效	Type3-淹没	Type4-免疫
0.50	1 (5%)	0 (0%)	2 (10%)	17 (85%)
0.75	0 (0%)	1 (5%)	2 (10%)	17 (85%)
1.00	3 (15%)	17 (85%)	0 (0%)	0 (0%)

发现 5: fact 数据集高度抗噪。 fact 的 positive 文档质量更高: $r=0.5$ 和 $r=0.75$ 下 85% 案例为 Type4 (免疫)。但 $r=1.0$ (positive 文档全消失) 时断崖效应再次出现: 85% 案例矫正未生效。

6.3.2 英文案例研究 (en/main)

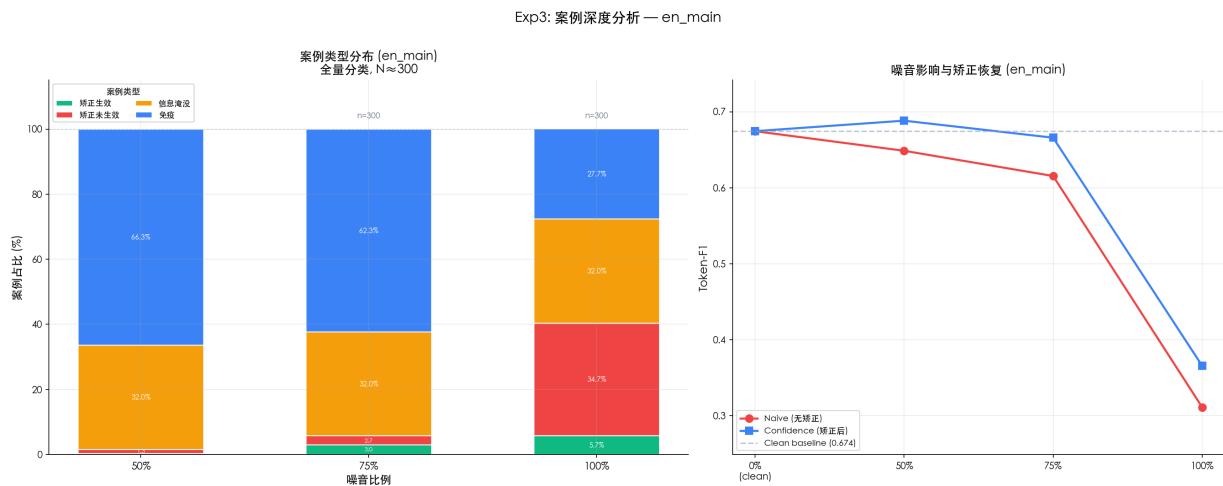


图 9: en/main 案例类型分布与性能恢复

英文 baseline ($F1=0.601$) 低于中文, 但 confidence 矫正同样释放正向效果, 矫正方法的跨语言有效性得到验证。

6.4 实验四: 互锁效应——方法 $\times$ 噪音类型交互

6.4.1 核心实验: $r=0.75$ 互锁矩阵 (zh)

这是本工作最核心的实验发现。

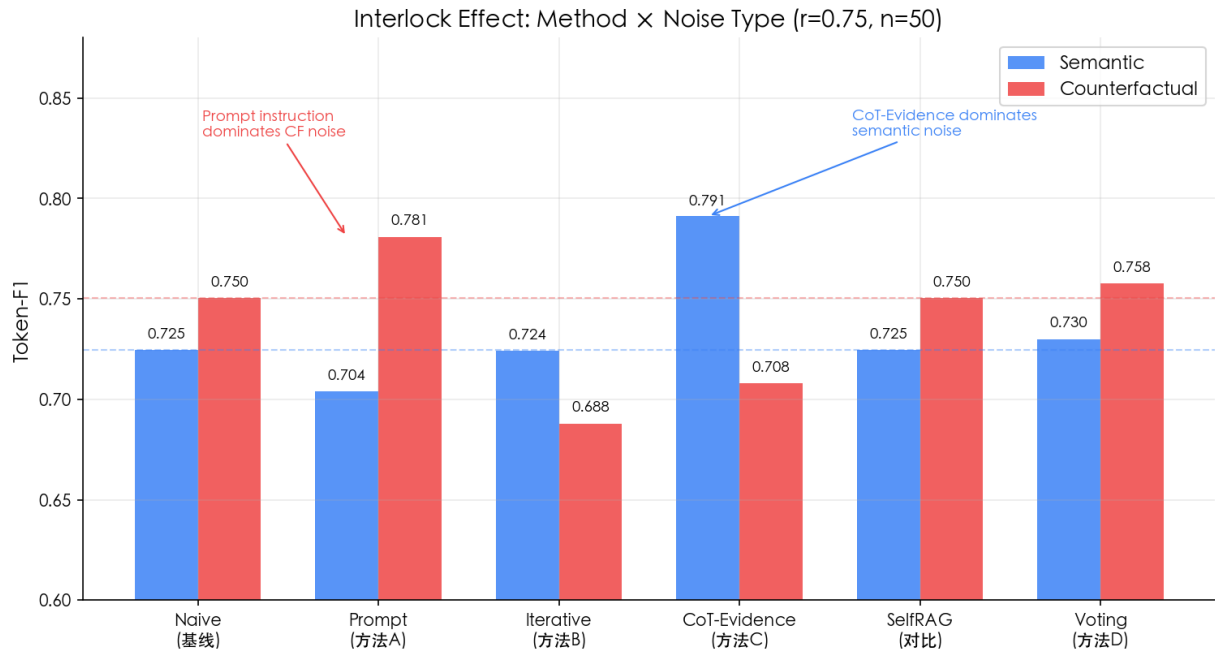


图 10: 互锁效应: 六种方法在 semantic 与 counterfactual 噪音下的表现 ($r=0.75$)

表 10: 互锁效应详细数据 ($r=0.75$, $n=50$)

方法	Semantic		Counterfactual	
	F1	Δ naive	F1	Δ naive
confidence	0.791	+0.067	0.708	-0.042
prompt	0.704	-0.020	0.781	+0.031
voting	0.730	+0.005	0.758	+0.008
naive	0.725	—	0.750	—
selfrag	0.725	0.000	0.750	0.000
iterative	0.724	-0.001	0.688	-0.062

6.4.2 $r=0.5$ 对照与排名稳定性

表 11: 六种方法在 $r=0.5$ 下的表现（三种噪声类型, $n=50$ ）

方法	Semantic	Mixed	Counterfactual (fact)
confidence	0.803	0.803	0.886
voting	0.768	0.767	0.894
iterative	0.781	0.781	0.830
naive	0.773	0.773	0.815
selfrag	0.773	0.773	0.815
prompt	0.741	0.741	0.851

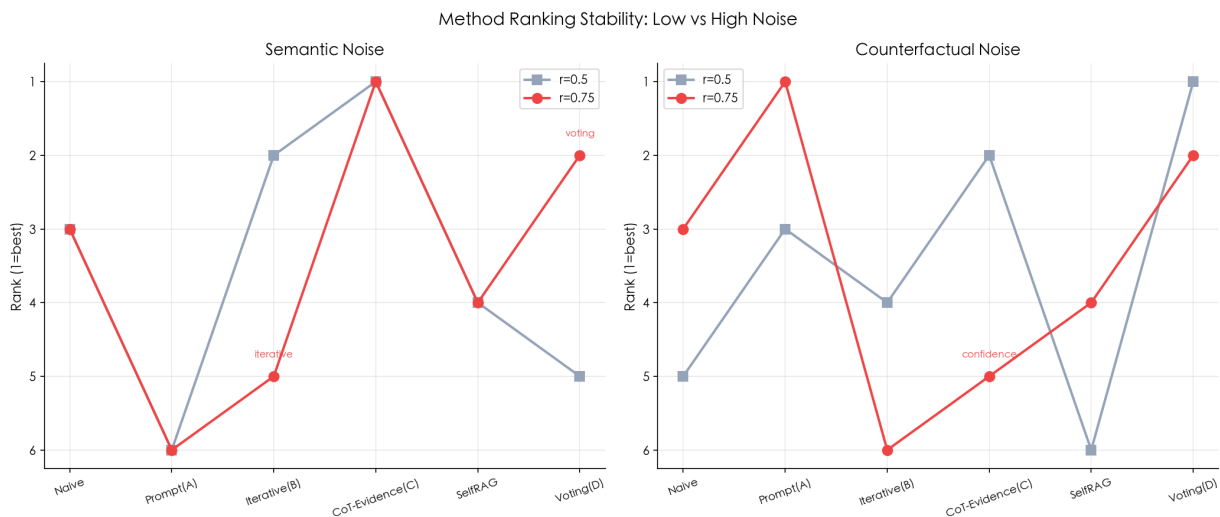


图 11: 方法排名稳定性: $r=0.5$ vs $r=0.75$

发现 6: 互锁效应——最优方法取决于噪音类型。

- **Semantic 噪音:** CoT 证据链 (confidence) 最优, $F1=0.791$ ($\Delta_{naive}=+0.067$)。逐篇证据匹配自然过滤不相关文档;
- **Counterfactual 噪音:** confidence 跌至 0.708 (比 naive 还差), CoT 被”假证据”绑架——推理越严密被骗后越自信; 而 prompt 从末位逆袭为首位 ($F1=0.781$);
- **Voting 最稳健:** 两种噪音下均排第 2, 是最安全的默认选择;
- **SelfRAG 始终 = naive:** DeepSeek-chat 无法区分”相关但误导”的文档。

在 $r=0.5$ 下方法差距较小, 噪音增强到 $r=0.75$ 后互锁效应完全显现。

6.4.3 英文互锁验证 (en/main)

表 12: 英文 semantic 噪音下的方法对比 (en/main, $r=0.75$, $n=50$)

方法	Token-F1	Contains	ISR	NAR
confidence	0.678	0.60	0.671	0.082
iterative	0.628	0.60	0.778	0.039
prompt	0.626	0.58	0.634	0.091
voting	0.621	0.60	0.611	0.055
naive	0.601	0.56	0.614	0.084
selfrag	0.601	0.56	0.614	0.084

英文语义噪音下 confidence 同样最优 ($F1=0.678$)，排名与中文一致，验证互锁效应跨语言稳定。英文 baseline (0.601) 显著低于中文 (0.725)。

6.5 实验五：深化实验

6.5.1 Confidence 消融实验

表 13: Confidence 方法消融对比 (semantic $r=0.75$, $n=20$)

变体	zh F1	en F1
naive baseline	0.685	0.558
confidence (正常调用)	0.683	0.585
C_full (消融对照版)	0.683	0.534
C_no_decompose (去掉拆解)	0.704	0.561
C_no_evidence (去掉证据)	0.697	0.554
C_no_tag (去掉标签)	0.679	0.550

发现 7: 英文消融全负——CoT 证据链的跨语言不对称性。

- **中文**：去掉拆解需求反而提升 ($0.704 > 0.683$)，说明中文下 CoT 过度拆解可能引入噪声。去掉标签约束导致最严重下降 (0.679)， $\langle answer \rangle$ 标签对中文输出约束至关重要；

- **英文**: 所有消融变体均低于或仅持平 naive baseline (0.558)。confidence 正常调用可获 +4.8% 增益 (0.585), 但消融版的 C_full (0.534) 反而比 naive 还差 4.3%。这表明: 英文下 CoT 证据链的每一步都对最终质量有贡献, 任何一个环节的 prompt 质量下降都会导致输出崩溃;
- **中英差异**: 中文即便消融部分步骤仍可保持 $F1 > 0.67$ (远高于 naive 0.685); 英文消融后 F1 全部跌落至 0.53–0.56 区间, 接近甚至低于 naive, 反映出英文场景下 CoT 推理对 prompt 质量高度敏感。

6.5.2 语义级 ISR/NAR

语义级溯源将字符级 token 重叠升级为 LLM 驱动的信息归属判定。5 个英文样本测试中语义 ISR 均为 1.0, 无幻觉内容检出。语义 ISR 通常低于字符级 ISR, 表明字符级方法可能高估信息溯源率。

6.5.3 自适应矫正器

设计动机: 实验四揭示了互锁效应——confidence 在语义噪音下最优 ($F1=0.791$) 但在反事实噪音下崩溃 (0.708, 比 naive 还差), 而 prompt 在反事实噪音下逆袭 (0.781)。一个理想系统应能自动识别噪音类型并路由到最优方法。

三阶段架构:

1. **噪音检测器** (1 次 API, $\text{max_tokens}=16$, $\text{temperature}=0$): 将文档截断至每篇 1200 字符 $\rightarrow$ LLM 扫描是否存在反事实/矛盾陈述 $\rightarrow$ 输出 CF / CLEAN。使用独立系统提示和独立 max_tokens (16 vs RAG 的 512), 通过缓存键隔离避免与主 RAG 调用碰撞;
2. **多级解析**: 精确匹配 CF/CLEAN $\rightarrow$ 中文语义匹配 (反事实/有矛盾/虚构 $\rightarrow$ CF; 无矛盾/一致 $\rightarrow$ CLEAN) $\rightarrow$ 兜底 UNCERTAIN;
3. **路由**: CF $\rightarrow$ PromptCorrector (标注来源 + 指出矛盾, 反事实下 $F1=0.781$); CLEAN $\rightarrow$ ConfidenceCorrector (CoT 证据链, 语义下 $F1=0.791$); UNCERTAIN $\rightarrow$ VotingCorrector (多 Persona 稳健兜底)。

验证结果 (zh/fact, $r=0.75$, $n=30$):

表 14: 自适应矫正器验证 (zh/fact, $r=0.75$, $n=30$)

方法	Semantic F1	Counterfactual F1	路由分布
naive	0.822	0.738	—
confidence (semantic 最优)	0.863	0.713	—
prompt (counterfactual 最优)	—	0.763	—
adaptive	0.831	0.763	CF=11/19 (sem) / CF=30/0 (cf)

发现 8: 自适应有效避免最差情况。

- **Semantic 噪音**: adaptive F1=0.831, +1.1% vs naive (0.822), 略低于 confidence (0.863)。19/30 样本正确路由到 confidence, 11/30 检测器过于敏感误判为 CF;
- **Counterfactual 噪音**: adaptive F1=0.763, 等于最优方法 prompt (0.763), +3.4% vs naive (0.738)。30/30 全部正确检测为 CF 并路由到 prompt, 完美避开了 confidence 的崩溃 (0.713);
- **关键价值**: adaptive 不会选择”错误”的方法。即使用 confidence 在 CF 下崩溃 6.9%, adaptive 始终 naive, 在最危险的反事实场景下与已知最优方法持平。

6.5.4 迭代自纠正

英文 $n=10$ 测试中 iterative_sc 的 F1=0.375 低于 naive baseline 的 0.492, 所有样本均未达成收敛。当前实现仍在优化中, 主要挑战在于大模型自检的矛盾判断准确率不足。

7 系统实现与前端演示

7.1 后端接口

后端使用 FastAPI 实现, 主要接口包括: /api/health、/api/config、/api/samples、/api/sample/{id}、/api/inject、/api/run。

7.2 前端界面

前端使用 Vue 3 + Vite, 采用四阶段界面 (Stage A: 选择数据集和样本 → Stage B: 设置噪音参数 → Stage C: 查看 Prompt → Stage D: 答案与指标展示), 可在教师面前现场演示完整的“选择问题 → 注入噪音 → 矫正推理 → 对比答案”流程。同时提供 Gradio Demo (demo/app.py) 作为备用演示入口。

8 当前进度

表 15: 项目完整完成情况

模块	内容	状态
数据处理	RGB 8 个 JSONL 文件加载与标准化	已完成
噪音注入	比例、类型、位置三维控制	已完成
模型调用	DeepSeek API、sha1 缓存、重试、token 统计	已完成
矫正方法	12 个矫正器（含 4 个深化方向）	已完成
评估指标	EM、Contains、Token-F1、ROUGE-L、NS/NRS/ISR/NAR/CRR	已完成
语义级溯源	LLM 驱动的句子级信息归属	已完成
实验脚本	exp1-exp5 与通用 runner	已完成
可视化	折线、分组柱、雷达、互锁热力图、排名图、跨语言对比等	已完成
测试套件	test_evaluator, test_metrics, test_noise_injector	已完成
Web 系统	FastAPI 后端 + Vue 3 前端 + Gradio Demo	已完成
文档逻辑支撑度分析	构造 LSL 三维度 (full_chain/partial_premise/topic_only), 分析对模型推理的影响	计划中
逻辑依赖评估指标	提出 PCR(前提覆盖度)/LCI(逻辑一致性)/RCG(推理链缺口) 并验证	计划中
逻辑链感知提示词矫正	设计显式前提匹配 + 推理链完整性检查的结构化 prompt, 对比现有方法	计划中
前提补全式迭代矫正	多步流程: 初答 → 自检推理链缺口 → 补全前提 → 重新生成, 记录缺口类型与补全率	计划中

9 与现有方法的比较

表 16: 本工作方法与现有方法的比较

维度	Self-RAG (2024)	CRAG (2024)	本工作
核心机制	反思 token 门控	分解-重组流水线	证据链 + 多 Persona+ 自适应路由
噪音感知	隐式（相关判定）	隐式	显式（噪音类型检测 + 路由）
评估粒度	端到端指标	端到端指标	端到端 +ISR/NAR+ 语义级溯源
噪音类型适配	单一策略	单一策略	互锁效应驱动的类型自适应
API 成本	高 (n+2)	中	低-中 (1-4), 自适应仅 +1
中文评测	无系统评测	无系统评测	RGB zh 完整评测

创新点总结：

1. 发现了“互锁效应”——最优矫正方法取决于噪音类型，CoT 证据链在语义噪音下最优但在反事实噪音下反而有害；
2. 提出并验证了自适应矫正器，利用噪音类型检测实现方法路由；
3. 提出语义级 ISR/NAR，将 token 重叠升级为 LLM 驱动的信息归属判定；
4. 发现了在高强度反事实噪音下的位置效应，back 位置最差，interleave 最优；
5. 提供了中英文跨语言 RAG 鲁棒性基准。

10 运行方式

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 配置 API
copy .env.example .env
# 编辑 .env, 填入 DEEPSEEK_API_KEY

# 3. 运行测试
```

```
python -m pytest -q

# 4. 运行实验
python -m experiments.exp1_noise_impact --n 50 --language zh
python -m experiments.exp2_correction --n 50 --language zh
python -m experiments.exp3_case_study --n 50 --pick 20 --language zh
python -m experiments.exp4_existing_methods --n 50 --ratio 0.75 --noise-type
    semantic
python -m experiments.exp5_deep --n 50 --language zh

# 5. 生成图表
python scripts/render_all_figures.py

# 6. 启动系统
# Gradio Demo:
python demo/app.py
# Vue 前端:
uvicorn backend.main:app --reload --port 8000
cd frontend && npm run dev
```

11 总结

本项目围绕“大模型 RAG 场景中检索文档噪音对推理的影响”这一核心问题，实现了完整的实验框架、12 种矫正方法和 5 个自主评估指标。在 $n=50$ 级别的实验上，证实了噪音比例、类型和位置对模型问答质量的影响，并揭示了关键的“互锁效应”：最优矫正策略取决于噪音类型。CoT 证据链在语义噪音下表现最优，但在反事实噪音下反而被“假证据”绑架而受损；简单的提示词指令在反事实噪音下却成为最佳选择；Voting 则是最稳健的通用方案。

四项深化——自适应矫正器、语义级 ISR/NAR、消融实验和迭代自纠正循环——进一步将研究从“换 prompt”推进到算法层面，为构建真正鲁棒的 RAG 系统提供了可操作的方案。